

Projeto Final de TR2 - Entrega 2

Streaming Adaptativo com Controle de Banda

Grupo 16

Leonardo Cortês – 200030582

Adriano Campos – 202061421

João Santos – 222025342

1 Descrição da Arquitetura Implementada

A arquitetura do cliente foi escrita em Python nativo e divide as responsabilidades em três frentes principais:

- **Cliente de Rede:** Responsável por ler o manifesto JSON e baixar os segmentos em blocos (*chunks*) de 4096 bytes via `urllib`, calculando assim o *throughput* (vazão instantânea) e a variação no atraso (*jitter*).
- **Buffer Manager:** Simula o comportamento do *playout buffer* de um dispositivo real, acumulando a duração cronológica dos segmentos recebidos. Ele subtrai o tempo gasto em I/O (tempo real de download), sinalizando eventos críticos caso a reserva zere. Adicionamos nesta versão um limite de "pacing" (`max.buffer = 10s`), forçando o cliente a aguardar caso a reserva ultrapasse o necessário.
- **Failover Controller:** Entidade reativa que, ao interceptar falhas de rota ou desconexões inesperadas de rede (`urllib.error.URLError`, `TimeoutError`), inspeciona a lista de servidores em ordem de prioridade. A disponibilidade é averiguada através de requisições de *health check* (GET /health) para consolidar a migração em tempo real.

2 Análise de Deficiências do Baseline

O algoritmo Baseline da Primeira Entrega operava sob a premissa puramente *Rate-Based*, calculando uma média móvel simples multiplicada por um fator de segurança de 0.8 para escolher a maior qualidade comportada matematicamente. A literatura e a especificação do projeto apontam uma deficiência teórica clássica nesta abordagem empírica: a propensão à **oscilação nervosa**. Em ambientes de rede reais com congestionamentos episódicos (comum no TCP devido ao tráfego rajado da Internet), o Baseline tende a alternar frequentemente entre resoluções, causando o que chamamos de *ping-pong*. Esse excesso de mudanças abruptas em questão de segundos piora ativamente a Qualidade de Experiência (QoE) do usuário.

3 Política 2: Rate-Based com Histerese

3.1 Motivação e Descrição

Para atacar o problema de oscilação, projetamos a **Política 2** encapsulada na classe `RateBasedHysteresisABR`. A nova abordagem aprimora a inteligência adaptativa de duas formas:

1. **Slow-Start Conservador:** A reprodução inicia na qualidade mais baixa possível (para maximizar o acumulo imediato no *playout buffer* antes de tomar riscos).
2. **Histerese com Confirmação:** Para alterar a qualidade, não basta o algoritmo ler um aumento isolado de vazão. A nova tendência de banda precisou ser avaliada e sustentada durante uma janela de N segmentos seguidos (parâmetro `confirm=3`). Somado a isso, o algoritmo é proibido de "pular" degraus (e.g. indo do $240p$ direto para o $1080p$), subindo ou caindo um único nível por vez.

3.2 Comparação com o Baseline

Ao expor a *Política 2* e o *Baseline* à mesma anomalia sintética na conexão, os resultados mostram uma curva de reatividade drasticamente diferente. Enquanto o *Baseline* acompanha cegamente a linha da rede e oscila frequentemente, a *Política 2* atua estabilizando a qualidade, ignorando quedas marginais.

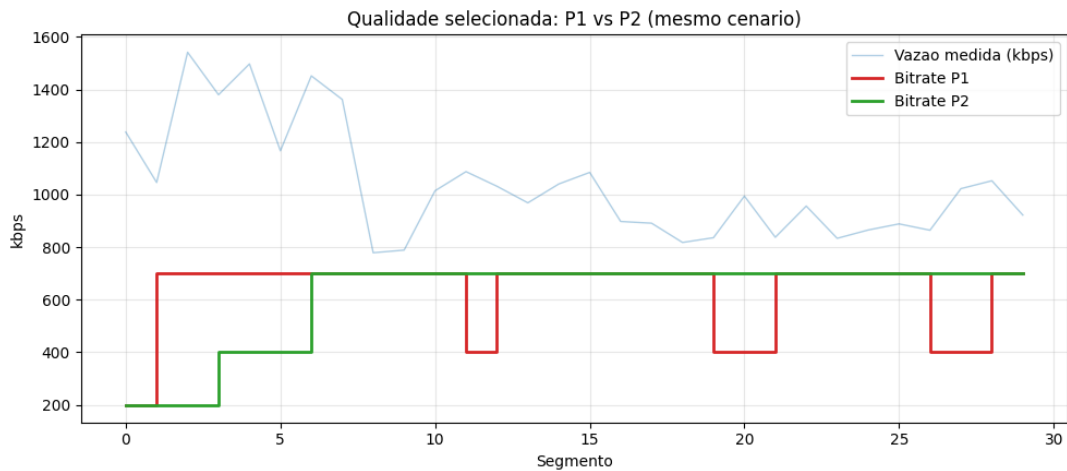


Figura 1: Comparativo de oscilação do Bitrate selecionado entre Baseline (P1) e Histerese (P2).

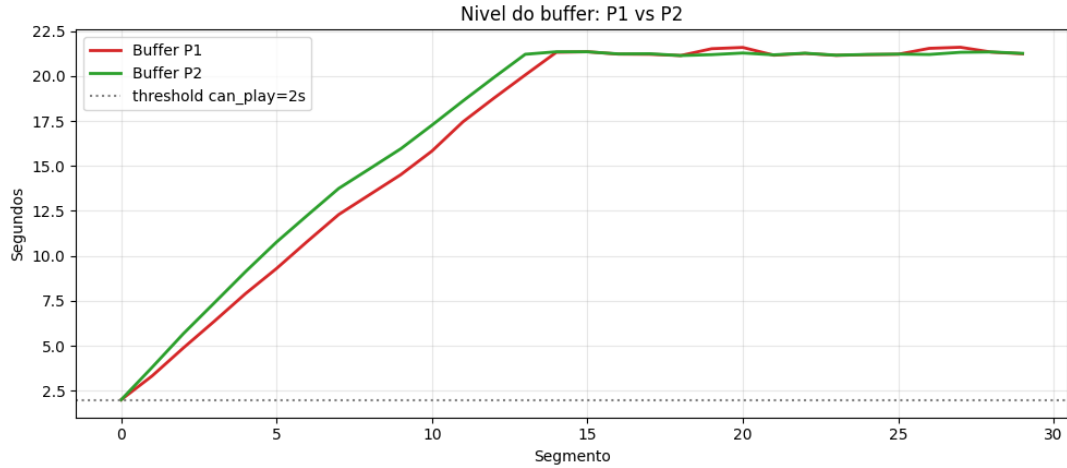


Figura 2: Comparativo do crescimento e estabilização do nível de Buffer entre as duas políticas.

4 Análise do Failover

O teste do Controlador de *Failover* revelou os seguintes comportamentos perfeitamente registrados em nossos arquivos de CSV:

- **Evento de Falha:** Durante o *download* do segmento 13, detectamos a falha do Servidor A (A queda foi precedida por uma notória queda no rendimento do transporte TCP local).
- **Nível de Buffer e Rebuffering:** O cliente demorou na faixa dos milissegundos para migrar para o Servidor B. O impacto na aplicação foi absorvido na sua totalidade graças a restrição do buffer no *pacing*. O log mostrava o nível do buffer em 21.22 segundos antes da troca; no instante pós-migração, o **buffer** perdeu margem irrisória e a flag *buffer_can_play* permaneceu 1 em toda a transição, blindando completamente o espectador contra o "congelamento de tela".
- **Decisão Pós-Falha:** Imediatamente após a queda para o *Servidor B*, a política ABR tentou preservar os **480p** que já mantinha, pois o buffer sustentava o tempo computacional. Contudo, pelo servidor reserva possuir latências maiores (consequência estabilizando em 850 kbps), no **segmento 17** o mecanismo de histerese atestou definitivamente que o enlace era inferior, resultando em um degrau unitário para trás, migrando graciosamente para **360p** e mantendo essa nova qualidade estacionária.

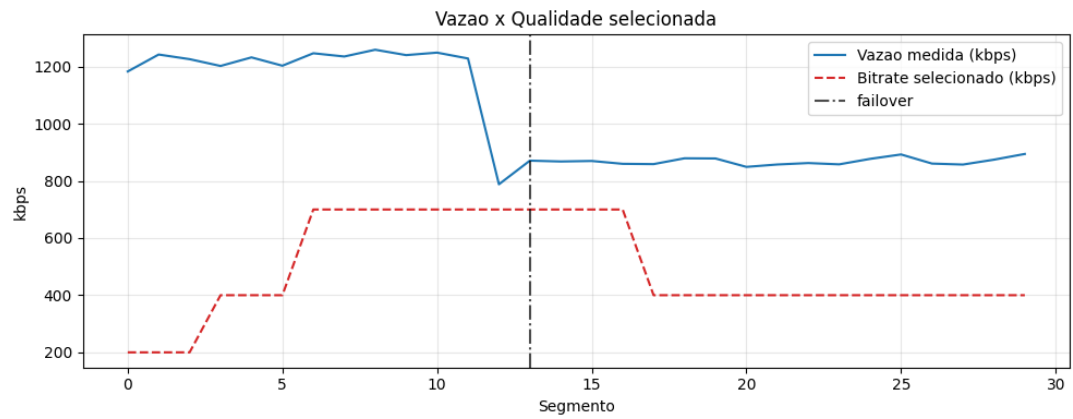


Figura 3: Vazão e Qualidade demonstrando a migração automática (linha vertical) para o Servidor B no segmento 13 e a adaptação do ABR no segmento 17.

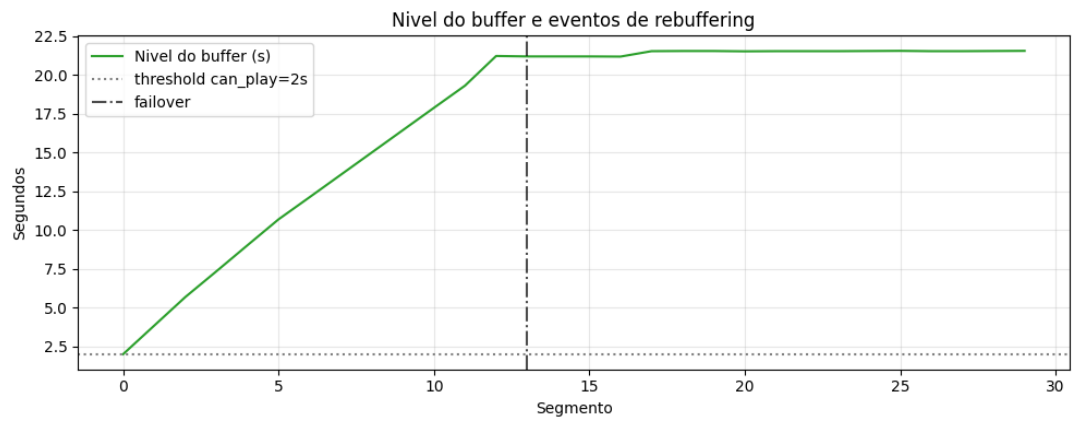


Figura 4: Nível do buffer permanecendo intacto (~21s) durante e após o evento de failover, evitando qualquer rebuffering.

```
(.venv) leopoldo@leonardo:~/unb/pf-tr2(main|+3...)$ .venv/bin/python experiment.py --mode all --segments 30 --outdir results

## FAILOVER (mock, derruba A apos 5.0s)
Manifest OK. policy=p2 servidores=['A', 'B'] qualidades=['240p', '360p', '480p', '720p', '1080p'] seg_dur=2.0s
Seg 0 | Srv=A | q= 240p | thr= 1220kbps | dl= 0.16s | buf= 2.00s | play=1 | jit= 5.7ms | stall=0.00s
Seg 1 | Srv=A | q= 240p | thr= 1151kbps | dl= 0.17s | buf= 3.83s | play=1 | jit= 4.3ms | stall=0.00s
Seg 2 | Srv=A | q= 240p | thr= 1190kbps | dl= 0.17s | buf= 5.66s | play=1 | jit= 5.8ms | stall=0.00s
Seg 3 | Srv=A | q= 360p | thr= 1197kbps | dl= 0.33s | buf= 7.32s | play=1 | jit= 2.6ms | stall=0.00s
Seg 4 | Srv=A | q= 360p | thr= 1199kbps | dl= 0.33s | buf= 8.99s | play=1 | jit= 2.8ms | stall=0.00s
Seg 5 | Srv=A | q= 360p | thr= 1208kbps | dl= 0.33s | buf=10.66s | play=1 | jit= 3.1ms | stall=0.00s
Seg 6 | Srv=A | q= 480p | thr= 1231kbps | dl= 0.57s | buf=12.09s | play=1 | jit= 2.1ms | stall=0.00s
Seg 7 | Srv=A | q= 480p | thr= 1193kbps | dl= 0.59s | buf=13.50s | play=1 | jit= 2.4ms | stall=0.00s
Seg 8 | Srv=A | q= 480p | thr= 1221kbps | dl= 0.57s | buf=14.93s | play=1 | jit= 2.0ms | stall=0.00s
Seg 9 | Srv=A | q= 480p | thr= 1185kbps | dl= 0.59s | buf=16.34s | play=1 | jit= 2.4ms | stall=0.00s
Seg 10 | Srv=A | q= 480p | thr= 1202kbps | dl= 0.58s | buf=17.76s | play=1 | jit= 2.4ms | stall=0.00s
>> derrubando servidor A (porta 8090)
Seg 11 | Srv=A | q= 480p | thr= 1166kbps | dl= 0.53s | buf=19.22s | play=1 | jit= 1.4ms | stall=0.00s
Falha em seg 12 no servidor A: <urlopen error [Errno 61] Connection refused>
>> FAILOVER seg 12: A -> B (1ms p/ achar servidor saudavel)
Seg 12 | Srv=B | q= 480p | thr= 860kbps | dl= 0.81s | buf=20.41s | play=1 | jit= 2.8ms | stall=0.00s
Seg 13 | Srv=B | q= 480p | thr= 873kbps | dl= 0.80s | buf=21.19s | play=1 | jit= 3.2ms | stall=0.00s
Seg 14 | Srv=B | q= 480p | thr= 861kbps | dl= 0.81s | buf=21.18s | play=1 | jit= 2.8ms | stall=0.00s
Seg 15 | Srv=B | q= 480p | thr= 858kbps | dl= 0.82s | buf=21.18s | play=1 | jit= 2.7ms | stall=0.00s
Seg 16 | Srv=B | q= 480p | thr= 860kbps | dl= 0.81s | buf=21.18s | play=1 | jit= 2.7ms | stall=0.00s
Seg 17 | Srv=B | q= 360p | thr= 855kbps | dl= 0.47s | buf=21.53s | play=1 | jit= 4.6ms | stall=0.00s
Seg 18 | Srv=B | q= 360p | thr= 859kbps | dl= 0.47s | buf=21.53s | play=1 | jit= 3.7ms | stall=0.00s
Seg 19 | Srv=B | q= 360p | thr= 855kbps | dl= 0.47s | buf=21.53s | play=1 | jit= 3.9ms | stall=0.00s
Seg 20 | Srv=B | q= 360p | thr= 848kbps | dl= 0.47s | buf=21.52s | play=1 | jit= 3.5ms | stall=0.00s
Seg 21 | Srv=B | q= 360p | thr= 851kbps | dl= 0.47s | buf=21.53s | play=1 | jit= 3.7ms | stall=0.00s
Seg 22 | Srv=B | q= 360p | thr= 841kbps | dl= 0.48s | buf=21.52s | play=1 | jit= 3.4ms | stall=0.00s
Seg 23 | Srv=B | q= 360p | thr= 861kbps | dl= 0.46s | buf=21.53s | play=1 | jit= 3.5ms | stall=0.00s
Seg 24 | Srv=B | q= 360p | thr= 846kbps | dl= 0.47s | buf=21.52s | play=1 | jit= 3.6ms | stall=0.00s
Seg 25 | Srv=B | q= 360p | thr= 881kbps | dl= 0.45s | buf=21.54s | play=1 | jit= 3.4ms | stall=0.00s
Seg 26 | Srv=B | q= 360p | thr= 858kbps | dl= 0.47s | buf=21.53s | play=1 | jit= 4.1ms | stall=0.00s
Seg 27 | Srv=B | q= 360p | thr= 869kbps | dl= 0.46s | buf=21.53s | play=1 | jit= 4.2ms | stall=0.00s
Seg 28 | Srv=B | q= 360p | thr= 852kbps | dl= 0.47s | buf=21.52s | play=1 | jit= 3.8ms | stall=0.00s
Seg 29 | Srv=B | q= 360p | thr= 872kbps | dl= 0.46s | buf=21.54s | play=1 | jit= 5.3ms | stall=0.00s
CSV gravado em results/failover/metrics.csv | rebuffers=0 failovers=1
failover @seg12: A->B em 1ms

-> Failover no segmento 12: server_id passou a B, qualidade 480p, buffer 20.41s, can_play=1 (1 = buffer absorveu a troca)
-> results/failover/throughput_quality.png
-> results/failover/buffer_level.png
```

Figura 5: Log de execução do cliente no terminal, evidenciando o momento em que ocorre a falha de conexão e o *FailoverController* atua migrando a requisição para o Servidor B.

5 Conclusão

Esta entrega valida empiricamente que a inclusão do conceito de Histerese na tomada de decisão (ABR) converte instabilidade sistêmica em resiliência de negócio. Os gráficos de desempenho demonstram ser vantajoso retardar uma melhoria visual para o usuário quando não se pode confirmar as premissas subjacentes da camada de enlace de antemão. Paralelamente, o tratamento programático das exceções de conexões provou que uma arquitetura bem configurada de "reserva de buffer" é o principal escudo contra a interrupção súbita de *Failovers*.